

Church Servants Management System (CSMS)

Phase 1 Data Modules & Models Design

Project: CSMS (Flutter)

Phase: Phase 1 (Foundation)

Date: January 2026

Architecture: Feature-First Clean Architecture

1. Overview

This document describes the **complete data models design** and **data modules organization** for CSMS Phase 1, covering:

- **Authentication (Auth)** feature with role-based user management
- **Servants Management** feature with list display and selection functionality

The design follows **Clean Architecture principles**, separating Domain, Data, and Presentation layers while adhering to **Composition over Inheritance** [1].

2. Core Design Principles

2.1 No Inheritance for Users

Instead of inheritance hierarchies (e.g., User, AdminUser, ServantUser), we use a **single UserModel/UserEntity** with an **Enum for roles** [1].

Rationale:

- Simpler to maintain and extend roles
- Reduces coupling between classes
- More aligned with Domain-Driven Design (enum as value object)

2.2 Composition Over Inheritance

All features compose lower-layer services via constructor injection [1].

Example:

```
// ✓ GOOD: Constructor injection
class AuthRepositoryImpl implements AuthRepository {
  final AuthRemoteDataSource remoteSource;
  final AuthLocalDataSource localSource;
```

```
  AuthRepositoryImpl({
    required this.remoteSource,
    required this.localSource,
```

```
});  
}
```

```
// ✗ AVOID: Extending datasources  
class AuthRepositoryImpl extends BaseRepository { ... }
```

2.3 Mappers for Model ↔ Entity Conversion

Each model type has a **dedicated mapper** to convert between Domain (Entity) and Data (Model) layers [1].

Example:

```
class UserMapper {  
  static UserEntity toDomain(UserModel model) {  
    return UserEntity(  
      id: model.id,  
      username: model.username,  
      role: UserRole.values[model.roleIndex],  
    );  
  }  
}
```

```
static UserModel toModel(UserEntity entity) {  
  return UserModel(  
    id: entity.id,  
    username: entity.username,  
    roleIndex: entity.role.index,  
  );  
}  
}
```

3. Complete File Structure

3.1 Project Root

```
csms/  
├─ lib/  
│   ├── main.dart  
│   └─ app/  
│       ├── app.dart  
│       ├── bootstrap/  
│       ├── di/  
│       ├── routing/  
│       └─ theme/  
├─ core/  
│   ├── constants/  
│   ├── error/  
│   ├── logging/  
│   ├── network/  
│   ├── storage/  
│   ├── firebase/  
│   └─ utils/
```

- └─ features/
 - └─ auth/
 - └─ servants/
- └─ test/
- └─ pubspec.yaml
- └─ analysis_options.yaml
- └─ firebase_options.dart

3.2 Core Module (lib/core)

- lib/core/
 - └─ constants/
 - └─ app_constants.dart
 - └─ class AppConstants {
 - static const appName = 'CSMS';
 - static const apiTimeout = Duration(seconds: 30);
 - └─ hive_type_ids.dart
 - └─ class HiveTypeIds {
 - static const int userModel = 0;
 - static const int servantModel = 1;
 - └─ error/
 - └─ exceptions.dart
 - └─ Custom exceptions:
 - FirebaseAuthException
 - FirestoreException
 - SecureStorageException
 - HiveException
 - NetworkException
 - └─ failures.dart
 - └─ Custom failures (for domain):
 - AuthFailure
 - ServantsFailure
 - NetworkFailure
 - └─ logging/
 - └─ app_logger.dart
 - └─ Uses dart:developer.log() instead of print()
 - └─ network/
 - └─ connectivity_service.dart
 - └─ Monitors network status for offline-first prep
 - └─ network_info.dart
 - └─ Interface for connectivity checking
 - └─ storage/
 - └─ secure/
 - └─ secure_storage_service.dart
 - └─ Wrapper for flutter_secure_storage

```

| | | Methods:
| | | - Future<void> write(String key, String value)
| | | - Future<String?> read(String key)
| | | - Future<void> delete(String key)
| | | - Future<void> deleteAll()
| | | └─ secure_keys.dart
| | | └─ class SecureKeys {
| | | static const userSessionKey = 'user_session';
| | | static const authTokenKey = 'auth_token';
| | | }
| | | └─ hive/
| | | └─ hive_service.dart
| | | └─ Manages Hive initialization & box registration
| | | Methods:
| | | - Future<void> init()
| | | - Box<T> getBox<T>(String name)
| | | - Future<void> clearAll()
| | | └─ hive_boxes.dart
| | | └─ class HiveBoxes {
| | | static const servantsBox = 'servants';
| | | static const userCache = 'user_cache';
| | | }
| | |
| | | └─ firebase/
| | | └─ firebase_initializer.dart
| | | └─ Firebase.initializeApp() + Firestore settings
| | |
| | | └─ utils/
| | | └─ validators.dart
| | | └─ Username/password/email validators
| | | └─ extensions.dart
| | | └─ Useful extensions (String, BuildContext, etc.)

```

3.3 Auth Feature (lib/features/auth)

lib/features/auth/

```

| | └─ domain/
| | └─ entities/
| | └─ user_entity.dart
| | dart | | class UserEntity { | | final String id; | | final String username; | | final
UserRole role; | | final DateTime createdAt; | | final DateTime? lastLogin; | | | | const
UserEntity({ | | required this.id, | | required this.username, | | required this.role, | |
required this.createdAt, | | this.lastLogin, | | }); | | } | |
| |
| | └─ enums/
| | └─ user_role.dart
| | dart | | enum UserRole { | | admin('Admin'), | | servant('Servant'); | | | | final
String displayName; | | const UserRole(this.displayName); | | } | |
| |
| | └─ repositories/

```

```

| | └─ auth_repository.dart
| | dart | | abstract class AuthRepository { | | Future<UserEntity> login(String username,
String password); | | Future<UserEntity?> getSession(); | | Future<void> logout(); | | } |
|
| | └─ usecases/
| | └─ login_usecase.dart
| | dart | | class LoginUseCase { | | final AuthRepository repository; | |
LoginUseCase(this.repository); | | | | Future<UserEntity> call(String username, String
password) { | | return repository.login(username, password); | | } | | } | |
| | └─ get_session_usecase.dart
| | dart | | class GetSessionUseCase { | | final AuthRepository repository; | |
GetSessionUseCase(this.repository); | | | | Future<UserEntity?> call() { | | return
repository.getSession(); | | } | | } | |
| | └─ logout_usecase.dart
| | dart | class LogoutUseCase { | final AuthRepository repository; |
LogoutUseCase(this.repository); | | Future<void> call() { | return repository.logout(); | } |
} |
|
| └─ data/
| └─ models/
| └─ user_model.dart
| | dart | | @JsonSerializable(fieldRename: FieldRename.snake) | | class UserModel { |
| final String id; | | final String username; | | final int roleIndex; // Index of UserRole
enum | | @JsonKey(name: 'created_at') | | final DateTime createdAt; | |
@JsonKey(name: 'last_login') | | final DateTime? lastLogin; | | | | const UserModel({ | |
required this.id, | | required this.username, | | required this.roleIndex, | | required
this.createdAt, | | this.lastLogin, | | }); | | | | factory UserModel.fromJson(Map<String,
dynamic> json) => | | _$UserModelFromJson(json); | | Map<String, dynamic> toJson() =>
_$UserModelToJson(this); | | } | |
| |
| | └─ datasources/
| | └─ auth_remote_data_source.dart
| | | | dart | | | abstract class AuthRemoteDataSource { | | | // Firebase Auth login | | |
Future<UserModel> login(String username, String password); | | | // Fetch user role from
Firestore if needed | | | Future<UserRole> fetchUserRole(String userId); | | | } | | | |
| | | class AuthRemoteDataSourceImpl implements AuthRemoteDataSource { | | | final
FirebaseAuth _firebaseAuth; | | | final Firestore _firestore; | | | |
AuthRemoteDataSourceImpl({ | | | required FirebaseAuth firebaseAuth, | | | required
Firestore firestore, | | | }): _firebaseAuth = firebaseAuth, _firestore = firestore; | |
| | | | @override | | | Future<UserModel> login(String username, String password)
async { | | | try { | | | // Step 1: Sign in with Firebase Auth (using email derived from
username) | | | final email = '$username@csms.church'; | | | final credential = await
_firebaseAuth.signInWithEmailAndPassword( | | | email: email, | | | password:
password, | | | ); | | | | | // Step 2: Fetch user role from Firestore | | | final userDoc =
await _firestore | | | .collection('users') | | | .doc(credential.user!.uid) | | | .get(); | | |
| | | final userData = userDoc.data(); | | | return UserModel( | | | id: credential.user!.uid,
| | | username: username, | | | roleIndex: userData['role_index'] as int? ?? 1, | | |
createdAt: DateTime.parse(userData['created_at'] as String), | | | lastLogin:
DateTime.now(), | | | ); | | | } on FirebaseAuthException catch (e) { | | | throw
FirebaseAuthException(message: e.message ?? 'Auth failed'); | | | } | | | } | | | } | |

```

```

| | |
| | └─ auth_local_data_source.dart
| | dart | | abstract class AuthLocalDataSource { | | Future<void>
saveSession(UserModel userModel); | | Future<UserModel?> getSession(); | |
Future<void> clearSession(); | | } | | | class AuthLocalDataSourceImpl implements
AuthLocalDataSource { | | final SecureStorageService _secureStorage; | | |
AuthLocalDataSourceImpl(this._secureStorage); | | | @override | | Future<void>
saveSession(UserModel userModel) async { | | final json =
jsonEncode(userModel.toJson()); | | await _secureStorage.write( | |
SecureKeys.userSessionKey, | | json, | | ); | | } | | | @override | | Future<UserModel?
> getSession() async { | | final json = await _secureStorage.read( | |
SecureKeys.userSessionKey, | | ); | | if (json == null) return null; | | return
UserModel.fromJson(jsonDecode(json)); | | } | | | @override | | Future<void>
clearSession() async { | | await _secureStorage.delete(SecureKeys.userSessionKey); | | } |
| } | |
| |
| └─ mappers/
| └─ user_mapper.dart
| | dart | | class UserMapper { | | static UserEntity toDomain(UserModel model) { | |
return UserEntity( | | id: model.id, | | username: model.username, | | role:
UserRole.values[model.roleIndex], | | createdAt: model.createdAt, | | lastLogin:
model.lastLogin, | | ); | | } | | | static UserModel toModel(UserEntity entity) { | |
return UserModel( | | id: entity.id, | | username: entity.username, | | roleIndex:
entity.role.index, | | createdAt: entity.createdAt, | | lastLogin: entity.lastLogin, | | ); | |
| | } | |
| |
| └─ repositories/
| └─ auth_repository_impl.dart
| | dart | class AuthRepositoryImpl implements AuthRepository { | final
AuthRemoteDataSource remoteDataSource; | final AuthLocalDataSource
localDataSource; | | AuthRepositoryImpl({ | required this.remoteDataSource, | required
this.localDataSource, | }); | | @override | Future<UserEntity> login(String username,
String password) async { | try { | // Attempt remote login | final userModel = await
remoteDataSource.login(username, password); | | // Cache locally | await
localDataSource.saveSession(userModel); | | // Return domain entity | return
UserMapper.toDomain(userModel); | } on Exception { | rethrow; | } | | } | | @override |
Future<UserEntity?> getSession() async { | final userModel = await
localDataSource.getSession(); | if (userModel == null) return null; | return
UserMapper.toDomain(userModel); | } | | @override | Future<void> logout() async { |
await localDataSource.clearSession(); | } | | }
|
| └─ presentation/
| └─ bloc/
| └─ auth_bloc.dart
| └─ auth_event.dart
| └─ auth_state.dart
| └─ pages/
| └─ login_page.dart
| └─ splash_page.dart
| └─ widgets/

```

```
| |—login_form.dart
| |—role_badge.dart
```

3.4 Servants Feature (lib/features/servants)

lib/features/servants/

```
|
|—domain/
| |—entities/
| | |—servant_entity.dart
| | dart | | class ServantEntity { | | final String id; | | final String fullName; | | final
String? phone; | | final String? email; | | final DateTime createdAt; | | final bool
isSelected; // For UI state | | | const ServantEntity({ | | required this.id, | | required
this.fullName, | | this.phone, | | this.email, | | required this.createdAt, | | this.isSelected
= false, | | }); | | } | |
| |
| |—repositories/
| | |—servants_repository.dart
| | dart | | abstract class ServantsRepository { | | Future<List<ServantEntity>>
fetchServants(); | | Future<void> cacheServants(List<ServantEntity> servants); | |
Future<List<ServantEntity>> getCachedServants(); | | } | |
| |
| |—usecases/
| | |—fetch_servants_usecase.dart
| | dart | | class FetchServantsUseCase { | | final ServantsRepository repository; | |
FetchServantsUseCase(this.repository); | | | Future<List<ServantEntity>> call() { | |
return repository.fetchServants(); | | } | | } | |
| | |—toggle_servant_selection_usecase.dart (UI logic - optional for domain)
| |
|—data/
| |—models/
| | |—servant_model.dart
| | dart | | @HiveType(typeId: 1) | | @JsonSerializable(fieldRename:
FieldRename.snake) | | class ServantModel { | | @HiveField(0) | | final String id; | | |
@HiveField(1) | | final String fullName; | | | @HiveField(2) | | final String? phone; | |
| | @HiveField(3) | | final String? email; | | | @HiveField(4) | | final DateTime
createdAt; | | | // Non-Hive field: UI state, not persisted | | @JsonKey(includeFromJson:
false, includeToJson: false) | | final bool isSelected; | | | const ServantModel({ | |
required this.id, | | required this.fullName, | | this.phone, | | this.email, | | required
this.createdAt, | | this.isSelected = false, | | }); | | | factory
ServantModel.fromJson(Map<String, dynamic> json) => | |
_$ServantModelFromJson(json); | | Map<String, dynamic> toJson() =>
_$ServantModelToJson(this); | | } | |
| |
| |—datasources/
| | |—servants_remote_data_source.dart
| | | dart | | | abstract class ServantsRemoteDataSource { | | |
Future<List<ServantModel>> fetchServants(); | | | } | | | class
ServantsRemoteDataSourceImpl implements ServantsRemoteDataSource { | | | final
FirebaseFirestore _firestore; | | | | ServantsRemoteDataSourceImpl(this._firestore); |
| | | | @override | | | Future<List<ServantModel>> fetchServants() async { | | | try {
```

```

    | | | final snapshot = await _firestore | | | .collection('servants') | | |
.orderBy('created_at', descending: true) | | | .get(); | | | | | return snapshot.docs | | |
.map((doc) => ServantModel.fromJson(doc.data())) | | | .toList(); | | | } on
FirebaseException catch (e) { | | | throw FirestoreException(message: e.message ??
'Firestore error'); | | | } | | | } | | | } | | |
| | |
| | | └─ servants_local_data_source.dart
| | dart | | abstract class ServantsLocalDataSource { | | Future<void>
cacheServants(List<ServantModel> servants); | | Future<List<ServantModel>>
getCacheServants(); | | } | | | class ServantsLocalDataSourceImpl implements
ServantsLocalDataSource { | | final HiveService _hiveService; | | |
ServantsLocalDataSourceImpl(this._hiveService); | | | @override | | Future<void>
cacheServants(List<ServantModel> servants) async { | | final box =
_hiveService.getBox<ServantModel>(HiveBoxes.servantsBox); | | await box.clear(); | |
await box.addAll(servants); | | } | | | | @override | | Future<List<ServantModel>>
getCacheServants() async { | | final box = _hiveService.getBox<ServantModel>
(HiveBoxes.servantsBox); | | return box.values.toList(); | | } | | | } | | |
| | |
| | └─ mappers/
| | └─ servant_mapper.dart
| | dart | | class ServantMapper { | | static ServantEntity toDomain(ServantModel
model) { | | return ServantEntity( | | id: model.id, | | fullName: model.fullName, | |
phone: model.phone, | | email: model.email, | | createdAt: model.createdAt, | |
isSelected: model.isSelected, | | ); | | } | | | static ServantModel toModel(ServantEntity
entity) { | | return ServantModel( | | id: entity.id, | | fullName: entity.fullName, | |
phone: entity.phone, | | email: entity.email, | | createdAt: entity.createdAt, | | isSelected:
entity.isSelected, | | ); | | } | | | static List<ServantEntity>
toDomainList(List<ServantModel> models) { | | return models.map((m) =>
toDomain(m)).toList(); | | } | | | } | | |
| | |
| | └─ repositories/
| | └─ servants_repository_impl.dart
| | dart | class ServantsRepositoryImpl implements ServantsRepository { | final
ServantsRemoteDataSource remoteDataSource; | final ServantsLocalDataSource
localDataSource; | final ConnectivityService connectivityService; | |
ServantsRepositoryImpl({ | required this.remoteDataSource, | required
this.localDataSource, | required this.connectivityService, | }); | | @override |
Future<List<ServantEntity>> fetchServants() async { | try { | // Strategy: Try remote first,
cache, then return | final remoteServants = await remoteDataSource.fetchServants(); | | //
Cache locally for offline-first | await localDataSource.cacheServants(remoteServants); | |
// Map to domain entities | return ServantMapper.toDomainList(remoteServants); | }
catch (e) { | // Fallback to cache if remote fails | final cachedServants = await
localDataSource.getCacheServants(); | if (cachedServants.isNotEmpty) { | return
ServantMapper.toDomainList(cachedServants); | } | rethrow; | } | } | | @override |
Future<void> cacheServants(List<ServantEntity> servants) async { | final models =
servants | .map((e) => ServantMapper.toModel(e)) | .toList(); | await
localDataSource.cacheServants(models); | } | | @override | Future<List<ServantEntity>>
getCacheServants() async { | final cachedModels = await
localDataSource.getCacheServants(); | return
ServantMapper.toDomainList(cachedModels); | } | } |
|

```



```

├─ presentation/
│   └─ bloc/
│       ├── servants_bloc.dart
│       ├── servants_event.dart
│       └── servants_state.dart
│   └─ pages/
│       └── servants_list_page.dart
│   └─ widgets/
│       ├── servant_list_tile.dart
│       └── servants_list_view.dart

```

4. Data Models Design Best Practices

4.1 UserModel Design

- **Role Storage:** Store role as int roleIndex (enum index) instead of string enum for JSON compatibility
- **Timestamps:** Always include createdAt for audit trails
- **Session Persistence:** Store minimal data locally (id, username, role, timestamps)
- **Null Safety:** Use DateTime? for optional timestamps like lastLogin

4.2 ServantModel Design

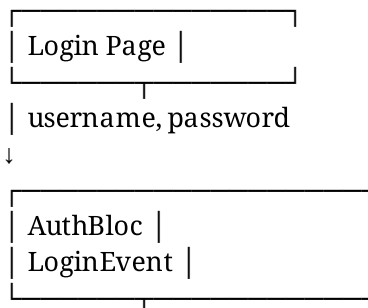
- **Hive Compatibility:** Use @HiveType and @HiveField annotations for local persistence
- **JSON Serialization:** Use @JsonSerializable with fieldRename: FieldRename.snake for API consistency
- **UI State Separation:** Keep isSelected transient (not persisted) by excluding it from JSON/Hive
- **Auditing:** Include createdAt timestamp for data integrity

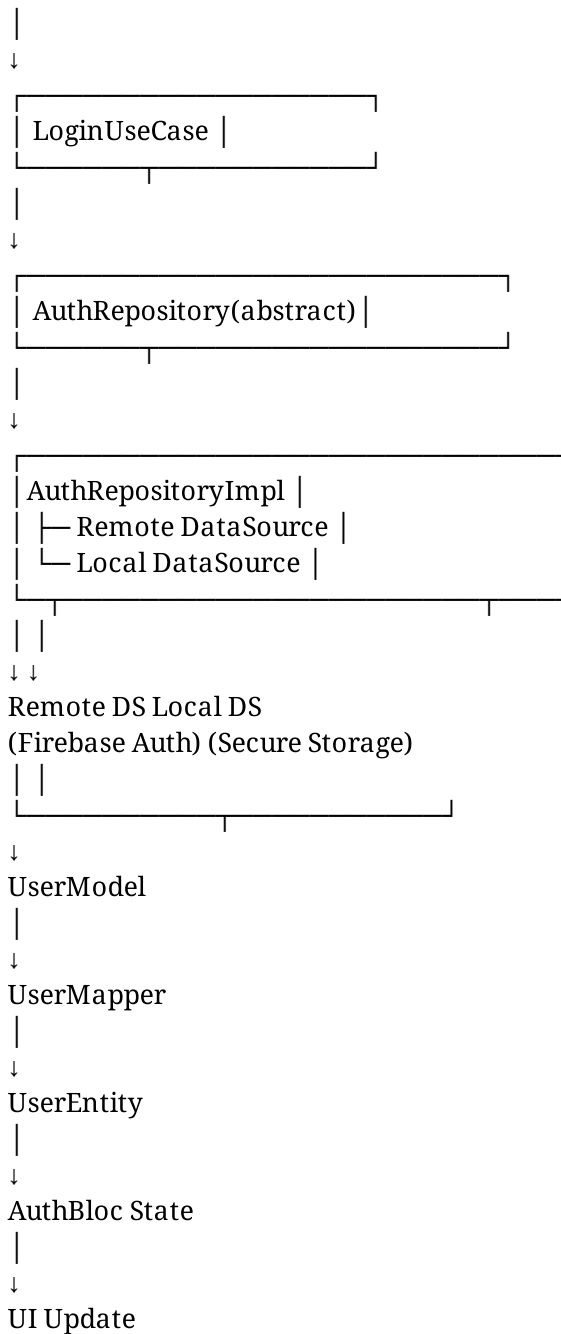
4.3 Mapper Responsibilities

- Convert between **Model** (data layer representation) and **Entity** (domain layer)
 - Handle **enum conversion** safely (use values[index] with bounds checking if needed)
 - Support **batch operations** (List<Model> -> List<Entity>)
 - Keep mappers **pure functions** (no side effects)
-

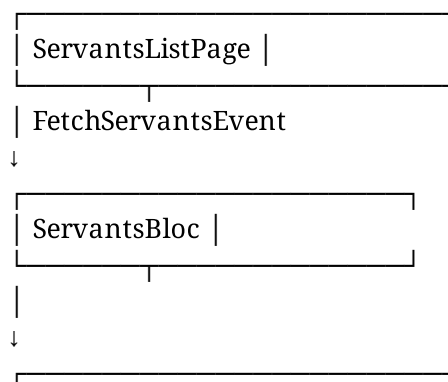
5. Data Flow Diagrams

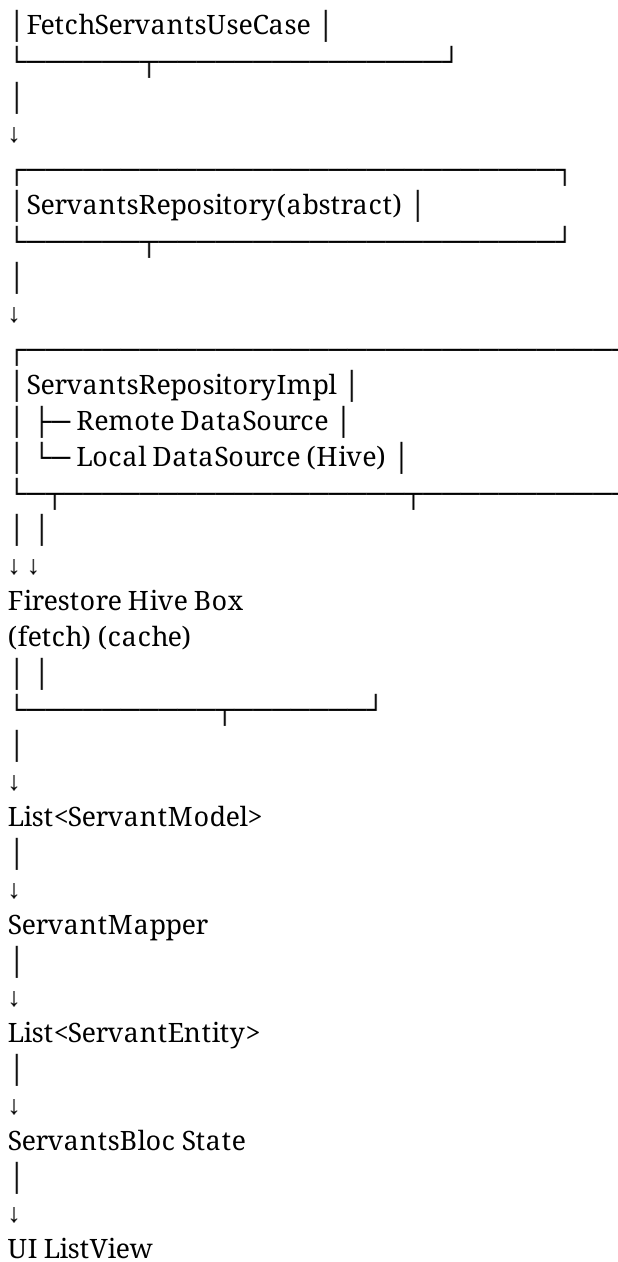
5.1 Authentication Flow





5.2 Servants Fetch Flow





6. Setup Instructions for Data Models

6.1 Add Dependencies

dependencies:

flutter:

sdk: flutter

firebase_core: latest

firebase_auth: latest

cloud_firestore: latest

flutter_secure_storage: latest

hive: latest

hive_flutter: latest

json_annotation: latest

```
dev_dependencies:  
build_runner: latest  
json_serializable: latest  
hive_generator: latest
```

6.2 Generate Serializable Code

After modifying models

```
dart run build_runner build --delete-conflicting-outputs
```

6.3 Initialize Firebase & Hive (in bootstrap)

```
// main.dart or app_bootstrap.dart  
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  // Initialize Firebase  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  
  // Initialize Hive  
  await Hive.initFlutter();  
  Hive.registerAdapter(ServantModelAdapter());  
  
  runApp(const MyApp());  
}
```

7. Key Points Summary

Aspect	Implementation
User Roles	Single UserModel + UserRole enum (no inheritance)
Data Persistence	Secure Storage (auth) + Hive (servants cache)
Remote Source	Firebase Auth + Firestore
Mappers	Dedicated UserMapper, ServantMapper for Model ↔ Entity
Offline-First	Try remote → cache → fallback to local on error
JSON Serialization	json_serializable with snake_case field names
Hive Caching	Type-safe adapters for ServantModel
Composition	Constructor injection, no inheritance
Error Handling	Custom exceptions/failures per layer
Testing Ready	Repositories are abstract, easy to mock

8. References

[1] Flutter Rules — Feature-First Clean Architecture & Composition over Inheritance. *AI Flutter Guidelines*.

[2] Firebase Flutter — Authentication & Firestore Documentation. *Firebase*.

[3] Hive — Lightweight key-value database for Flutter. *Hive Documentation*.

[4] JSON Serialization in Dart. *Dart & Flutter Official Docs*.

End of Document

CSMS Phase 1 Data Modules & Models Design
January 2026